Sidecar Pattern

Category: Cross-cutting concerns **One-liner:** Attach a helper process alongside each service instance to handle concerns like logging, metrics, security, and networking — so the service itself contains only business logic.

Part 0 — Concepts you need first

What is a "cross-cutting concern"?

Functionality needed by **many parts of a system** but not part of any one part's core purpose.

Your Order Service exists to manage orders. That's its **business logic** (the code that implements what the business actually does). But it also needs to: log what it does, emit **metrics** (numeric measurements like request counts and response times), **authenticate** callers (verify who they are), **encrypt** traffic (scramble it so only the intended receiver can read it), retry failed calls, and report **traces** (records of a request's path through every service it touched). **Every service needs those**, and none of them is about orders.

Cross-cutting concerns are a problem because the obvious solutions are both bad:

- **Copy the code into every service** → duplication, and changes require touching everything
- **Put it in a shared library** (a reusable code package every service imports) → better, but see the polyglot problem below

What is a process?

A running program with its own memory space. Two processes are genuinely separate — one crashing doesn't corrupt the other's memory.

What is a container?

A package containing an application plus everything it needs to run — libraries, **configuration** (settings supplied from outside the code), the lot. It runs isolated from other containers on the same machine. **Docker** is the common tool for building them.

What is a Kubernetes Pod?

The smallest deployable unit in **Kubernetes** (the standard platform for running containers across a cluster of machines). Crucially, **a Pod can contain more than one container**, and containers in the same Pod:

- Are always **scheduled onto the same machine** together (**scheduling** = Kubernetes choosing which machine runs something)
- Share a **network namespace** (one private, isolated set of network addresses and ports) — they can talk to each other over `localhost` (the special address meaning "this same machine")
- Can share **storage volumes** (directories of files mounted into the containers)
- Are **created and destroyed together**

This is the mechanism that makes the Sidecar pattern practical. Without co-scheduling and shared networking, a "helper alongside the service" would just be another remote service.

What is a proxy?

Software that sits between two parties and relays traffic. A **transparent** proxy relays traffic without the application knowing it exists — which is exactly how production sidecars work.

What is a service mesh?

An infrastructure layer handling all service-to-service communication. Concretely, it's **a sidecar proxy deployed next to every service**, plus a central **control plane** (the management component that pushes settings to every proxy) that configures them all. **Istio** (using **Envoy**, a high-performance open-source proxy) is the best-known example.

The polyglot problem — why shared libraries aren't enough

Polyglot means using many programming languages. Suppose you write an excellent library handling retries, metrics, and **mTLS** (mutual TLS — both sides of a connection present certificates to prove their identity, and the traffic is encrypted). Now your company has services in Java, Go, Python, and Node.

You must write and maintain **four versions**. They will drift. A bug fixed in the Java one lingers in Python for months. Adding a fifth language means a fifth implementation before that team can ship.

A sidecar solves this because it's a separate process speaking network protocols (standard rules for exchanging data, such as HTTP). It doesn't care what language the service beside it is written in.

Part 1 — The metaphor and the core idea

The name comes from a **motorcycle sidecar**: attached to and operating alongside the motorcycle, sharing the journey but remaining a distinct unit. The course also offers Batman and Robin, or Sherlock Holmes and Dr Watson — an aide that handles specific tasks, operates autonomously, and yet is part of a single larger entity.

The pattern encapsulates **modularity and separation of concerns** (building a system from independent pieces, each responsible for one job): deploy components of an application as **separate processes running alongside the main application**. Each sidecar is dedicated to a specific function and provides its service to the main application.

This lets developers **encapsulate the logic and complexity of those functions in the sidecar**, reducing the complexity of the main application.

The strength is its simplicity and modularity. Keeping each function isolated in its own environment makes it easier to develop, manage, and scale each component independently. That gives cleaner, more maintainable code — and makes it possible to **use different technologies for different tasks within the same application**.

⚠️ Part 2 — One correction to the course material

The course states that the sidecar and main application "run in the same process" and are "deployed together in the same container." **That is not how this works in practice**, and since you're using these notes as your reference, it's worth having right:

- The sidecar runs as a **separate process, in its own container**
- Both containers are **co-located in the same Kubernetes Pod** (or equivalent scheduling unit)
- They **share the Pod's network namespace** — hence `localhost` communication — and can share volumes
- They **share a lifecycle** — created, scheduled, and destroyed together

Same-process or same-container would defeat the isolation that makes the pattern valuable in the first place: if the sidecar shared a process with the app, a sidecar crash would take the app down, and you couldn't use a different language for each.

The course's other framing — **same lifespan, shares resources, communicates locally** — is correct. Everything else below holds.

Part 3 — The problem: monolithic application management

Lack of isolation

In a **monolith** (an application built and deployed as one single unit) all components are closely intertwined, so **a failure in one component can bring down the entire application**. The course's image: all your eggs in one basket — if the basket falls, all the eggs break. This lack of **fault isolation** (containing a failure so it can't spread) makes monoliths vulnerable and risky.

Scalability — moving the mountain

Scaling a monolith is like moving a mountain. **You cannot scale a particular component or functionality; you must scale the entire application**, even if only one component needs it. This leads to unnecessary resource consumption and inefficient scaling.

Complexity — a tangled web

As a monolith grows, so does its complexity. More features mean the **codebase** (all the source code of the application) becomes a tangled web that's increasingly difficult to understand, manage, or modify. Maintenance becomes a nightmare and development slows significantly.

Technology constraint — one size does not fit all

With a monolith you're largely **stuck with the technology stack chosen at the beginning** (the **stack** is the set of languages, frameworks and databases used). Want to integrate a new technology or switch to a more efficient one? That's a mammoth task. This lack of flexibility is a significant constraint given how rapidly technology evolves.

Part 4 — The solution

Pair each service with a secondary component — a **sidecar** — that takes over some of the main service's responsibilities. The two run alongside each other and communicate directly, creating a self-contained module that's easy to manage and scale.

Flexibility. By separating the main service from its secondary functions, it becomes easier to update, modify, or replace individual components without affecting the others. To change a specific piece of functionality, you **update the relevant sidecar and leave the rest of the application untouched** — no full **redeploy** (rebuilding and releasing the whole application).

Enhanced scalability. Since each sidecar is a separate entity, you can scale each independently. Surge in user logins? Scale up the **authentication sidecar**. High load on data processing? Scale up the **data-processing sidecar**. This is more efficient and saves resources.

Fault isolation. Each sidecar is independent, so a failure in one does not affect the others. Separate baskets for each egg — if one falls, the rest are safe.

Reduced complexity. Breaking the application into self-contained modules, each with its own sidecar, makes it easier to understand, manage, and maintain. Like clearing a path through dense forest — you can finally see where you're going.

Technology freedom. The independent, flexible structure makes it possible to **use different technologies for different sidecars**, letting you take advantage of the latest advances rather than being locked in.

Part 5 — Architecture

Co-located deployment

The main application and its sidecar are deployed **together**, sharing resources and communicating efficiently. This shared environment lets them **operate as a single unit while maintaining their individuality**.

Multiple sidecars

You are not restricted to one sidecar per application. You can have **several, each addressing a specific functionality**, letting you modularise the application.

> A motorbike with several sidecars — one for extra storage, one for a passenger, one carrying navigation aids. Each serves a distinct purpose; together they make one smooth ride.

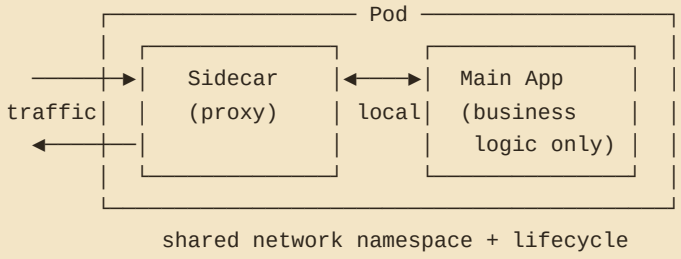
The network navigator — the sidecar as proxy

This is the most important architectural role, and the one that makes service meshes work.

The sidecar **intercepts all incoming and outgoing network traffic** from the main application, effectively acting as a **proxy**. This means it can handle:

- **Load balancing** (spreading requests across several instances of a service — see System Design Fundamentals)
- **Circuit breaking** (stopping calls to a service that keeps failing) *(see Circuit Breaker)*
- **Security** — including mutual TLS
- **Service discovery** (finding where other services are currently running) *(see Service Discovery)*

...leaving the main application **free to focus on delivering the primary service**.



> **Why "intercepts" is the key word.** The application doesn't call the sidecar and doesn't know it exists. It makes what it thinks is a normal **HTTP** call (an ordinary web request); traffic is transparently routed through the sidecar. That's what allows you to add mTLS, retries, and **tracing** (recording each request's path across services) to an application **without changing a line of its code** — even for an application whose source you can't easily modify.

Shared lifespan

The sidecar and main application, while separate entities, **share the same lifespan** — created, deployed, and destroyed together. This ensures the main application **always has its sidecar at its side**, ready to serve.

That guarantee matters. If the sidecar could be scheduled elsewhere or restarted independently, the application would need to handle its absence — reintroducing exactly the complexity you were

removing.

Part 6 — Worked example

The main application — a Spring Boot REST service

(**Spring Boot** is a popular Java framework for building web services quickly; a **REST** service answers plain HTTP requests on URLs such as `/api`.)

```
@SpringBootApplication
@RestController
public class MainApplication {

    public static void main(String[] args) {
        SpringApplication.run(MainApplication.class, args);
    }

    @GetMapping("/api")
    public String getData() {
        return "Hello, Sidecar!";
    }
}
```

A simple web service returning a string whenever `/api` is hit (each such URL is an **endpoint**).

The sidecar — a logging helper

```
@SpringBootApplication
@RestController
public class LoggingSidecar {

    public static void main(String[] args) {
        SpringApplication.run(LoggingSidecar.class, args);
    }

    @GetMapping("/log")
    public String logRequest() {
        System.out.println("Request received at /api endpoint");
        return "Logged!";
    }
}
```

This sidecar prints a log message every time `/log` is hit. Note it's also a Java application here — but that's incidental, and in practice it usually wouldn't be.

Wiring them together

```
@GetMapping("/api")
public String getData() {
    ResponseEntity<String> response =
        restTemplate.getForEntity("http://localhost:8081/log", String.class);
    System.out.println("Response from sidecar: " + response.getBody());
    return "Hello, Sidecar!";
}
```

`RestTemplate` is Spring's utility for calling RESTful services. Now whenever `/api` is hit, the main application calls the sidecar's `/log` endpoint, the sidecar logs the request, and the main application continues processing.

Notice `localhost:8081` — that's the shared network namespace at work. The main app doesn't need service discovery to find its sidecar; it's always right there at a known local port.

> ⚠️ **This example is pedagogically useful but architecturally backwards.** The main application *explicitly calling* the sidecar defeats the central benefit — the application now knows about the sidecar, contains code **coupling** it to the sidecar (making it depend on the sidecar's details), and would break if the sidecar changed. Real sidecars are **transparent**, intercepting traffic without the application's knowledge (as Istio and Envoy do below). Read this as "how two co-located processes communicate," not as the intended usage pattern.

The deployment factor

The course is explicit that the Java example doesn't cover deployment. In production you would:

- Use a container platform like **Docker** to package the main application and sidecar
- Use **Kubernetes** to **orchestrate** deployment and scaling (automatically start, place, restart and scale the containers)

Kubernetes has **built-in support for sidecar containers**, making it the natural platform for implementing this pattern.

Part 7 — Performance implications

"Does adding a sidecar to my application slow things down?" A legitimate question. It's a trade-off worth understanding rather than dismissing.

Resource cost

Each sidecar is an **additional process** running alongside your main application. That means more **CPU** (processor time), more memory, more network **bandwidth** (data-transfer capacity).

Individually, the sidecar's resource usage may not be substantial. **But multiply that by the number of instances of your application.** With hundreds or thousands of instances each carrying its own sidecar, **cumulative resource usage becomes significant**.

> Made concrete: a sidecar using 100 MB of memory across 1,000 pods is 100 GB of RAM spent on infrastructure rather than application work. That's a real budget line, and it's the most common objection to adopting a service mesh.

Latency

Having a sidecar introduces an **additional network hop** (one more trip between two programs over the network), adding **latency** (delay per request), since requests may go through the sidecar before reaching the main application or an external service.

The latency introduced is **usually negligible** — it's a **loopback** connection (traffic that never leaves the machine) within the same Pod, not a real network traversal. But in **high-performance, low-latency systems this could become an issue**, and it's worth measuring rather than assuming.

The verdict

The slight performance impact often **pales in comparison to the benefits**. The sidecar enhances functionality and maintainability and improves the overall development and operational experience. You need to balance these trade-offs for your situation.

Part 8 — Special considerations

1. Sidecar and main application versioning

Versioning becomes more complex. You must manage versions of **both** the main application and the sidecar. What if a new sidecar version is **incompatible with an older application version**, or vice versa? How do you **roll out** updates (release them gradually) across a **fleet** (all the running instances)? These questions need answering before you adopt the pattern, not after.

2. Shared resources

Sidecars share resources with the main application — **network and disk**. It's important the sidecar **doesn't hog these resources**, leaving nothing for the main application. Manage this using **resource limits and quotas at the container level** (hard caps on the CPU and memory each container may use).

> Without limits, a misbehaving sidecar — say, one **buffering** logs (holding them in memory to send later) during an **outage** (period of unavailability) of the log collector — can starve the very application it's meant to support.

3. Security

Sidecars often need **similar security permissions to the main application** to perform their tasks. This **increases the attack surface** (the total set of places an attacker could try to break in) and needs careful consideration when designing your sidecar.

A sidecar handling mTLS holds **private keys** (the secret half of an encryption key pair; whoever has it can impersonate the service). A sidecar collecting logs can read application output. Compromising the sidecar can be as valuable to an attacker as compromising the application.

4. Complexity

Sidecars solve many problems, but they **also add complexity**. Developers and operators must now understand and manage **multiple parts instead of a single application**. Debugging a request means reasoning about the application *and* the proxy in front of it.

> The pattern is not a one-size-fits-all solution. Like any architectural pattern it has strengths and weaknesses, and its effectiveness depends on your specific use case.

Part 9 — Use cases

Monitoring and metrics collection

You're running an application with multiple instances and want metrics from each. **Embedding monitoring code in each application would make things complex.** A sidecar handles metrics collection, leaving your application code clean and focused on core functionality.

Logging

A sidecar can **collect logs from the main application and forward them to a central logging system**. This abstracts away logging complexity, making the application easier to manage — and means changing your log destination doesn't require touching any application.

Security and authentication

Instead of embedding security and authentication logic into your main application, **delegate it to a sidecar**. This simplifies the application and makes security measures **more consistent and easier to update** — you patch one sidecar image rather than auditing dozens of services.

Networking features

Networking is complex. A sidecar can handle **service discovery, connection pooling, circuit-breaking, intelligent routing** (**connection pooling** = keeping open connections ready for reuse instead of opening a new one per request), and more, letting your application focus on business logic without worrying about networking intricacies.

> The unifying test: **is this concern identical across all your services, and unrelated to what any of them actually does?** If yes, it's a sidecar candidate.

Part 10 — System design examples

Microservices architecture

Each microservice, as a main application, can have a sidecar providing **ancillary** (supporting, secondary) services such as logging, monitoring, and service discovery.

> Picture a fleet of ships, each ship a microservice. The sidecars are little boats accompanying each ship, helping with navigation, communication, and maintenance — while the ships focus on their main task.

Kubernetes and Istio — the canonical example

Istio is a service mesh for Kubernetes, and it uses this pattern extensively. Istio deploys a **sidecar proxy — Envoy — alongside each service** to:

- Handle **all network communication**
- **Enforce policies**
- **Collect telemetry data** (metrics, logs and traces sent automatically from running software)

> Picture a busy highway system where each car (service) has a dedicated assistant (the Envoy sidecar) for navigation, communication, and safety — enabling the driver to concentrate on driving.

This is the pattern at its purest. The application code contains **zero** networking, retry, mTLS, or **observability** logic (code that makes the system's internal behaviour visible to operators); all of it lives in infrastructure, configured centrally and applied uniformly.

Part 11 — Sidecar vs. shared library

The real decision most teams face:

	Shared library	Sidecar
Languages	One per language, maintained separately	One, language-agnostic
Upgrade	Rebuild and redeploy every service	Update the sidecar image
Runtime cost	Negligible — same process	An extra process per instance
Latency	None	A local hop
Failure isolation	A library bug crashes the app	Sidecar can fail separately
Best when	One language, few services, latency-critical	Polyglot, many services, frequent policy changes

Rule of thumb: with a single language and a handful of services, a library is simpler and cheaper. With several languages, many services, or a need to change cross-cutting behaviour without redeploying everything, the sidecar earns its overhead.

Part 12 — Common beginner misconceptions

"The sidecar and app are the same container." No — separate containers in the same Pod. See Part 2.

"The app calls the sidecar." In the course's toy example, yes. In real deployments, no — the sidecar transparently intercepts traffic. That transparency is the whole value.

"A sidecar is a microservice." No. A **microservice** (a small, independently deployed service) serves many clients and owns a **domain** (an area of the business, such as orders or payments). A sidecar serves exactly **one** application instance and owns no domain — its lifecycle is bound to its host.

"Sidecars are only for service meshes." Meshes are the most famous use, but **log shippers** (programs that forward log files to a central store), metrics **agents** (small background collectors), config reloaders, and **secret-fetchers** (programs that fetch passwords and keys from a vault) are all sidecars.

"It's free because it's just localhost." It costs one process worth of CPU and memory **per instance**, which at scale is the main reason teams hesitate.

"You need Kubernetes." Kubernetes makes it easy with Pods, but the pattern predates it — any system that can co-schedule two processes with shared networking works.

Part 13 — Check your understanding

1. What makes a concern a "cross-cutting" one?
2. Why doesn't a shared library solve cross-cutting concerns in a polyglot organisation?
3. What Kubernetes feature makes the pattern practical, and which two properties of it matter most?
4. Why is "intercepts" a more important word than "calls"?
5. What's the main objection to adopting a service mesh at scale?
6. When is a shared library the better choice?

(Answers: 1 — it's needed by many components but isn't the core purpose of any of them; 2 — you'd need a separate implementation per language, and they inevitably drift; 3 — the Pod, because it co-

schedules containers on one machine and gives them a shared network namespace and lifecycle; 4 — interception means the application needs no knowledge or code changes, which is what lets you add mTLS and tracing to services you can't modify; 5 — cumulative resource cost, one extra process per instance across thousands of instances; 6 — a single language, few services, and latency-critical paths.)

Part 14 — Interview soundbite (30 seconds)

> "The sidecar pattern attaches a helper process alongside each service instance to handle cross-cutting concerns — logging, metrics, auth, networking — so the service itself contains only business logic. It's co-located: a separate container in the same Kubernetes Pod, sharing the network namespace and lifecycle, so they communicate over localhost. The key role is acting as a proxy that transparently intercepts all inbound and outbound traffic, which lets it do load balancing, circuit breaking, mTLS, and service discovery without the application knowing it exists. That transparency is the point — you can add mTLS and distributed tracing to a service without changing a line of its code. The canonical example is Istio, where an Envoy sidecar runs next to every service and the app has no networking code at all. The big advantage over a shared library is that it's language-agnostic, so a polyglot estate gets consistent behaviour and you can upgrade policy without redeploying every service. The costs are real though: one extra process per instance multiplies across thousands of pods, so a sidecar using 100 MB across a thousand pods is 100 GB of RAM; there's an extra local hop; you now version the sidecar against the app; and the sidecar needs permissions that widen your attack surface."

Part 15 — Implementation

A. Runnable demo

Full source: `demos/SidecarDemo.java`

```
cd demos && ./gradlew runCh09
```

The whole point in two class definitions. Before:

```
String handleRequest(String payload) {
    requestCount.incrementAndGet();                    // metrics
    System.out.println("[app] logging inbound request"); // logging
    if (!payload.startsWith("signed:")) return "401";    // security
    String result = businessLogic(payload);              // <- the only real work
    // ... emit latency metric
}
```

After:

```
static class CleanApp {
    String handleRequest(String payload) {
        return "order-created:" + payload;               // business logic ONLY
    }
}
```

Everything removed from the app now lives in the sidecar, which **intercepts** rather than being called:

```
String handleInbound(String rawRequest, String clientCert) {
    if (!"valid-cert".equals(clientCert)) {
        return "403 Forbidden";                          // app never invoked
    }
    String response = app.handleRequest(rawRequest);      // plain call, no plumbing
    // ... record latency
}
```

B. What you should see

```
=== 3. mTLS enforced by the sidecar ===
  -> 403 Forbidden
  >> rejected before the application was ever invoked

=== 4. Retry + backoff, also in the sidecar ===
    [sidecar] upstream failed (connection reset), retrying
    [sidecar] upstream ok on attempt 3
  upstream was actually called 3 times; the app asked once
  >> CleanApp contains no retry code whatsoever

=== 5. Telemetry collected without any app instrumentation ===
      requests_ok        1
      requests_rejected  1
      upstream_retries   2

=== 7. The cost: resource multiplier ===
      1,000 pods x 100MB sidecar = 100,000 MB (97.7 GB) of RAM
      5,000 pods x 100MB sidecar = 500,000 MB (488.3 GB) of RAM
```

Section 4 is the pattern in one line: the **upstream** (the service being called) was called three times; the application asked once. Retry, backoff, and failure handling happened entirely outside the code you wrote.

Section 7 is the honest counterweight — at a thousand pods you're spending ~98 GB of RAM on infrastructure. That's the number that shows up in the meeting where someone asks whether the mesh is worth it.

C. Production — Istio

Injection — label the **namespace** (here a Kubernetes namespace: a named group of resources within a cluster) and every pod gets an Envoy sidecar automatically. `kubectl` is the Kubernetes command-line tool:

```
kubectl label namespace production istio-injection=enabled
```

Your **Deployment** (the Kubernetes object that describes which containers to run and how many copies) is unchanged; the sidecar appears at **admission time** (the moment Kubernetes accepts the object, when it can modify it):

```
spec:
  containers:
    - name: order-service          # your app - you wrote this
      image: order-service:1.4.2
    # - name: istio-proxy          # injected automatically, you did NOT write this
```

mTLS across the whole mesh — no application code, no **certificate** handling (a certificate is a signed file proving an identity):

```yaml
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: production
spec:
  mtls:
    mode: STRICT       # every service-to-service call is encrypted and authenticated
```

Retries, timeouts, and circuit breaking as configuration: (A `VirtualService` sets routing rules; a `DestinationRule` sets traffic policy for a destination. **Outlier detection** temporarily *ejects* — stops routing to — instances that keep returning errors.)

```yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: inventory
spec:
  hosts: [inventory]
  http:
    - timeout: 3s
      retries:
        attempts: 3
        perTryTimeout: 1s
        retryOn: 5xx,reset,connect-failure
      route:
        - destination: { host: inventory }
---
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: inventory
spec:
  host: inventory
  trafficPolicy:
    connectionPool:
      tcp:  { maxConnections: 100 }
      http: { http1MaxPendingRequests: 50, maxRequestsPerConnection: 10 }   # a bulkhead
    outlierDetection:                       # this IS a circuit breaker
      consecutive5xxErrors: 5
      interval: 10s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
```

> Read that last block carefully: **you just configured Bulkhead, Retry, and Circuit Breaker in YAML, for every service, in every language, without touching application code.** That is the argument for a service mesh in a single **manifest** (a YAML file describing desired Kubernetes resources).

Non-mesh sidecars you'll meet just as often (**Fluent Bit** is a lightweight log shipper; the Cloud SQL proxy opens a secure **tunnel** — an encrypted pass-through connection — to a Google Cloud database):

```yaml
containers:
  - name: app
    image: my-app:1.0
    volumeMounts: [{ name: logs, mountPath: /var/log/app }]
  - name: log-shipper                    # sidecar: tails and ships logs
    image: fluent-bit:2.2
    volumeMounts: [{ name: logs, mountPath: /var/log/app }]
  - name: cloud-sql-proxy                # sidecar: secure DB tunnel
    image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2.8
volumes:
  - name: logs
    emptyDir: {}                         # shared between the two containers
```

Kubernetes native sidecars (1.29+) — solves the classic startup/shutdown ordering problem (an **initContainer** normally runs to completion before the app starts; `restartPolicy: Always` turns it into a long-running sidecar):

```yaml
initContainers:
  - name: istio-proxy
    image: istio/proxyv2
    restartPolicy: Always        # <- makes it a true sidecar: starts first, stops last
```

D. Implementation pitfalls

1. The app starting before the sidecar is ready. The classic failure: your app makes its first outbound call, the proxy isn't up, and it fails at startup. Use native sidecars (`restartPolicy: Always` on an initContainer), or `holdApplicationUntilProxyStarts: true` in Istio.

2. The sidecar stopping before the app finishes. The mirror image at shutdown — **in-flight** requests (ones already started but not finished) fail because the proxy died first. Native sidecars fix this too.

3. No resource limits on the sidecar. An unbounded proxy can starve the application it exists to serve. Always set them (`100m` = one tenth of a CPU core; `128Mi` = 128 mebibytes of memory):

```yaml
annotations:
  sidecar.istio.io/proxyCPU: "100m"
  sidecar.istio.io/proxyMemory: "128Mi"
```

4. Adopting a full mesh for three services. The resource multiplier and operational complexity are fixed costs. Below roughly a dozen services in one or two languages, a shared library is cheaper and simpler. Be able to justify the mesh.

5. Forgetting the sidecar is in the network path when debugging. A 503 (the HTTP status code for "service unavailable") may come from Envoy, not your service. Learn to read the proxy's logs and its response flags before blaming application code.

6. Duplicating retries in app and mesh. If your HTTP client retries 3× and the mesh retries 3×, you get 9 requests. Pick one layer — usually the mesh — and disable the other.

7. Assuming mTLS means you're done with authorization. mTLS proves *who* is calling. It does not say *what they may do* — that's **authorization**. You still need `AuthorizationPolicy` resources.

Related patterns

- **Service Discovery** — meshes absorb discovery into the sidecar entirely

- **Circuit Breaker**, **Retry**, **Bulkhead** — all commonly enforced in the sidecar rather than in application code
- **API Gateway** — the gateway handles north-south (external, into and out of the system) traffic; the mesh of sidecars handles east-west (internal, service-to-service)
- **Config Externalization** — a sidecar can fetch config and expose it locally to the app